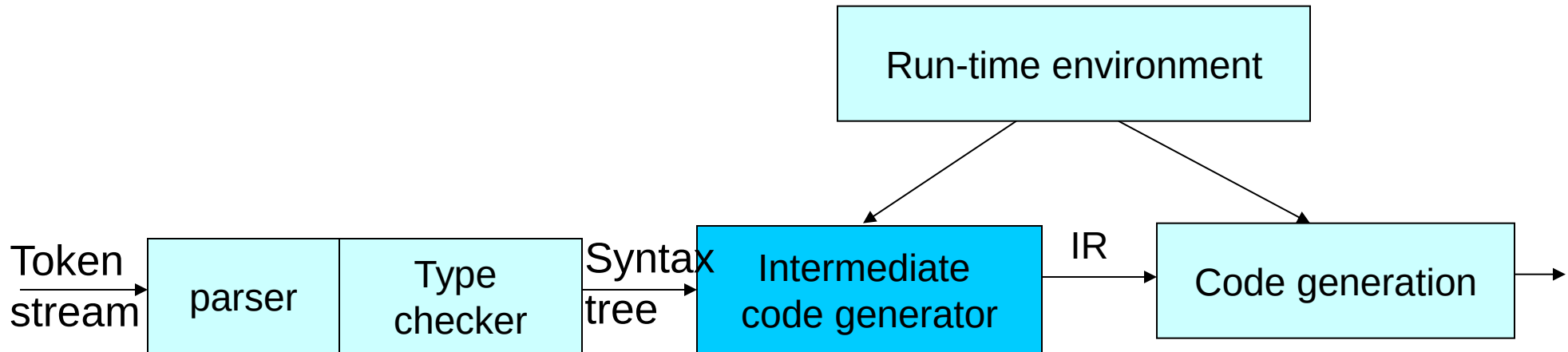


Intermediate Code Generation (P.463)

- Intermediate codes are machine independent codes, but they are close to machine instructions.
- The given program in a source language is converted to an equivalent program in an intermediate language by the intermediate code generator.



Intermediate language

- ◆ Intermediate language can be many different languages, and the designer of the compiler decides this intermediate language.
 - syntax trees can be used as an intermediate language.
 - postfix notation can be used as an intermediate language.
 - three-address code (Quadruples) can be used as an intermediate language
 - we will use quadruples to discuss intermediate code generation
 - quadruples are close to machine instructions, but they are not actual machine instructions.
 - some programming languages have well defined intermediate languages.
 - java – java virtual machine
 - prolog – warren abstract machine
 - In fact, there are byte-code emulators to execute instructions in these intermediate languages.

Three-Address Code (Quadraples)

- A quadraple is:

$$x := y \text{ op } z$$

where x , y and z are names, constants or compiler-generated temporaries; op is any operator.

- But we may also the following notation for quadraples (much better notation because it looks like a machine code instruction)

$$\text{op } x, y, z$$

apply operator op to y and z , and store the result in x .

- We use the term “three-address code” because each statement usually contains three addresses (two for operands, one for the result).

Three-Address Statements

Binary Operator: $\text{op result}, y, z$ or $\text{result} := y \text{ op } z$

where op is a binary arithmetic or logical operator. This binary operator is applied to y and z, and the result of the operation is stored in result.

Ex: add a,b,c
 addi a,b,c
 gt a,b,c

Unary Operator: $\text{op result}, y$ or $\text{result} := \text{op } y$

where op is a unary arithmetic or logical operator. This unary operator is applied to y, and the result of the operation is stored in result.

Ex: uminus a,,c
 not a,,c
 inttoreal a,,c

Three-Address Statements (cont.)

Move Operator: `mov result,,y` or `result := y`

where the content of `y` is copied into `result`.

Ex: `mov a,,c`
 `movi a,,c`
 `movr a,,c`

Unconditional Jumps: `jmp ,,L` or `goto L`

We will jump to the three-address code with the label `L`, and the execution continues from that statement.

Ex: `jmp ,,L1` `// jump to L1`
 `jmp ,,7` `// jump to the statement 7`

Three-Address Statements (cont.)

Conditional Jumps: `jmprelop y,z,L` or
if *y relop z* goto L

We will jump to the three-address code with the label L if the result of *y relop z* is true, and the execution continues from that statement. If the result is false, the execution continues from the statement following this conditional jump statement.

Ex:

<code>jmpgt y,z,L1</code>	// jump to L1 if $y > z$
<code>jmpge y,z,L1</code>	// jump to L1 if $y \geq z$
<code>jmpeq y,z,L1</code>	// jump to L1 if $y == z$
<code>jmpne y,z,L1</code>	// jump to L1 if $y \neq z$

Our relational operator can also be a unary operator.

<code>jmpnz y,,L1</code>	// jump to L1 if y is not zero
<code>jmpz y,,L1</code>	// jump to L1 if y is zero
<code>jmpt y,,L1</code>	// jump to L1 if y is true
<code>jmpf y,,L1</code>	// jump to L1 if y is false

Three-Address Statements (cont.)

Procedure Parameters: param x,, or param x

Procedure Calls: call p,n, or call p,n

where x is an actual parameter, we invoke the procedure p with n parameters.

Ex: param x_1 ,,
param x_2 ,,

 $\rightarrow p(x_1, \dots, x_n)$

param x_n ,,
call p, n ,

```
f(x+1,y) ➔      add  t1,x,1
                  param t1,,
                  param y,,
                  call f,2,
```

Three-Address Statements (cont.)

Indexed Assignments:

move x,,y[i] or $x := y[i]$

move y[i],,x or $y[i] := x$

Address and Pointer Assignments:

moveaddr x,,y or $x := \&y$

movecont x,,y or $x := *y$

Syntax-Directed Translation into Three-Address Code (P.468)

$S \rightarrow id := E$	$S.code = E.code \parallel gen('mov' \ id.place \ ',,' \ E.place)$
$E \rightarrow E_1 + E_2$	$E.place = newtemp();$ $E.code = E_1.code \parallel E_2.code \parallel gen('add' \ E.place \ ',E_1.place \ ', E_2.place)$
$E \rightarrow E_1 * E_2$	$E.place = newtemp();$ $E.code = E_1.code \parallel E_2.code \parallel gen('mult' \ E.place \ ', E_1.place \ ', E_2.place)$
$E \rightarrow - E_1$	$E.place = newtemp();$ $E.code = E_1.code \parallel gen('uminus' \ E.place \ ',,' \ E_1.place)$
$E \rightarrow (E_1)$	$E.place = E_1.place;$ $E.code = E_1.code$
$E \rightarrow id$	$E.place = id.place;$ $E.code = null$

Syntax-Directed Translation (cont.)

$S \rightarrow \text{while } E \text{ do } S_1$	<pre>S.begin = newlabel(); S.after = newlabel(); S.code = gen(S.begin ":") E.code gen('jmpf' E.place ',', S.after) S₁.code gen('jmp' ',', S.begin) gen(S.after ':')</pre>
$S \rightarrow \text{if } E \text{ then } S_1 \text{ else } S_2$	<pre>S.else = newlabel(); S.after = newlabel(); S.code = E.code gen('jmpf' E.place ',', S.else) S₁.code gen('jmp' ',', S.after) gen(S.else ':') S₂.code gen(S.after ':')</pre>

Declarations (P.473)

$P \rightarrow M D$

$M \rightarrow \epsilon \quad \{ \text{offset}=0 \}$

$D \rightarrow D ; D$

$D \rightarrow \text{id} : T \quad \{ \text{enter}(\text{id.name}, T.\text{type}, \text{offset}); \text{offset}=\text{offset}+T.\text{width} \}$

$T \rightarrow \text{int} \quad \{ T.\text{type}=\text{int}; T.\text{width}=4 \}$

$T \rightarrow \text{real} \quad \{ T.\text{type}=\text{real}; T.\text{width}=8 \}$

$T \rightarrow \text{array}[\text{num}] \text{ of } T_1 \quad \{ T.\text{type}=\text{array}(\text{num.val}, T_1.\text{type});$
 $\quad \quad \quad T.\text{width}=\text{num.val} * T_1.\text{width} \}$

$T \rightarrow \uparrow T_1 \quad \{ T.\text{type}=\text{pointer}(T_1.\text{type}); T.\text{width}=4 \}$

where *enter* creates a symbol table entry with given values.

Nested Procedure Declarations (P.476)

- For each procedure we should create a symbol table.

`mktable(previous)` – create a new symbol table where `previous` is the parent symbol table of this new symbol table

`enter(symtable,name,type,offset)` – create a new entry for a variable in the given symbol table.

`enterproc(symtable,name,newsymtable)` – create a new entry for the procedure in the symbol table of its parent.

`addwidth(symtable,width)` – puts the total width of all entries in the symbol table into the header of that table.

- We will have two stacks:
 - `tblptr` – to hold the pointers to the symbol tables
 - `offset` – to hold the current offsets in the symbol tables in `tblptr` stack.

Nested Procedure Declarations (P.477)

$P \rightarrow M D \quad \{ \text{addwidth}(\text{top}(\text{tblptr}), \text{top}(\text{offset})); \text{pop}(\text{tblptr}); \text{pop}(\text{offset}) \}$

$M \rightarrow \epsilon \quad \{ t = \text{mktable}(\text{nil}); \text{push}(t, \text{tblptr}); \text{push}(0, \text{offset}) \}$

$D \rightarrow D ; D$

$D \rightarrow \text{proc id } N D ; S$
 $\quad \{ t = \text{top}(\text{tblptr}); \text{addwidth}(t, \text{top}(\text{offset}));$
 $\quad \quad \text{pop}(\text{tblptr}); \text{pop}(\text{offset});$
 $\quad \quad \text{enterproc}(\text{top}(\text{tblptr}), \text{id.name}, t) \}$

$D \rightarrow \text{id} : T \{ \text{enter}(\text{top}(\text{tblptr}), \text{id.name}, T.\text{type}, \text{top}(\text{offset}));$
 $\quad \quad \text{top}(\text{offset}) = \text{top}(\text{offset}) + T.\text{width} \}$

$N \rightarrow \epsilon \quad \{ t = \text{mktable}(\text{top}(\text{tblptr})); \text{push}(t, \text{tblptr}); \text{push}(0, \text{offset}) \}$

Translation Scheme to Produce Three-Address Code – Assignment Statements (P.478)

$S \rightarrow id := E$	{ p= lookup(id.name); if (p is not nil) then emit('mov' p',,' E.place) else error("undefined-variable") }
$E \rightarrow E_1 + E_2$	{ E.place = newtemp(); emit('add' E.place ',,' E ₁ .place ',,' E ₂ .place) }
$E \rightarrow E_1 * E_2$	{ E.place = newtemp(); emit('mult' E.place ',,' E ₁ .place ',,' E ₂ .place) }
$E \rightarrow - E_1$	{ E.place = newtemp(); emit('uminus' E.place ',,' E ₁ .place) }
$E \rightarrow (E_1)$	{ E.place = E ₁ .place; }
$E \rightarrow id$	{ p= lookup(id.name); if (p is not nil) then E.place = id.place else error("undefined-variable") }

Translation Scheme with Locations

$S \rightarrow id := \{ E.inloc = S.inloc \} E$

 { $p = \text{lookup}(id.name)$;

 if (p is not nil) then { $\text{emit}(E.outloc \text{ 'mov' } E.place \text{ ',,' } p)$; $S.outloc = E.outloc + 1$ }

 else { $\text{error}(\text{"undefined-variable"})$; $S.outloc = E.outloc$ } }

$E \rightarrow \{ E_1.inloc = E.inloc \} E_1 + \{ E_2.inloc = E_1.outloc \} E_2$

 { $E.place = \text{newtemp}()$; $\text{emit}(E_2.outloc \text{ 'add' } E.place \text{ ',,' } E_1.place \text{ ',,' } E_2.place)$; $E.outloc = E_2.outloc + 1$ }

$E \rightarrow \{ E_1.inloc = E.inloc \} E_1 * \{ E_2.inloc = E_1.outloc \} E_2$

 { $E.place = \text{newtemp}()$; $\text{emit}(E_2.outloc \text{ 'mult' } E.place \text{ ',,' } E_1.place \text{ ',,' } E_2.place)$; $E.outloc = E_2.outloc + 1$ }

$E \rightarrow - \{ E_1.inloc = E.inloc \} E_1$

 { $E.place = \text{newtemp}()$; $\text{emit}(E_1.outloc \text{ 'uminus' } E.place \text{ ',,' } E_1.place)$; $E.outloc = E_1.outloc + 1$ }

$E \rightarrow (E_1) \{ E.place = E_1.place; E.outloc = E_1.outloc + 1 \}$

$E \rightarrow id \{ E.outloc = E.inloc; p = \text{lookup}(id.name)$;

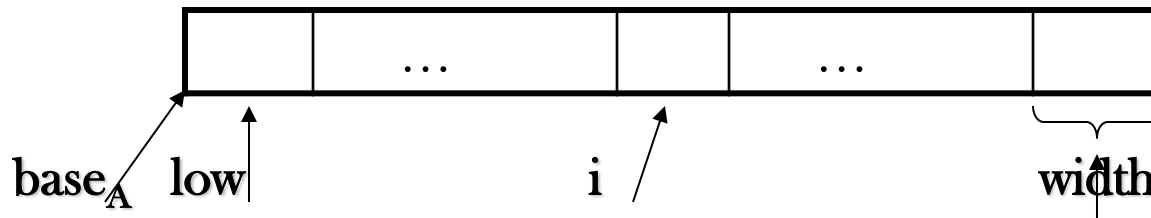
 if (p is not nil) then $E.place = id.place$

 else $\text{error}(\text{"undefined-variable"})$ }

Arrays (P.481)

- Elements of arrays can be accessed quickly if the elements are stored in a block of consecutive locations.

A one-dimensional array A:



$base_A$ is the address of the first location of the array A,

width is the width of each array element.

low is the index of the first array element

location of $A[i] \rightarrow base_A + (i - low) * width$

Arrays (cont.)

$\text{base}_A + (i - \text{low}) * \text{width}$

can be re-written as $i * \text{width} + (\text{base}_A - \text{low} * \text{width})$

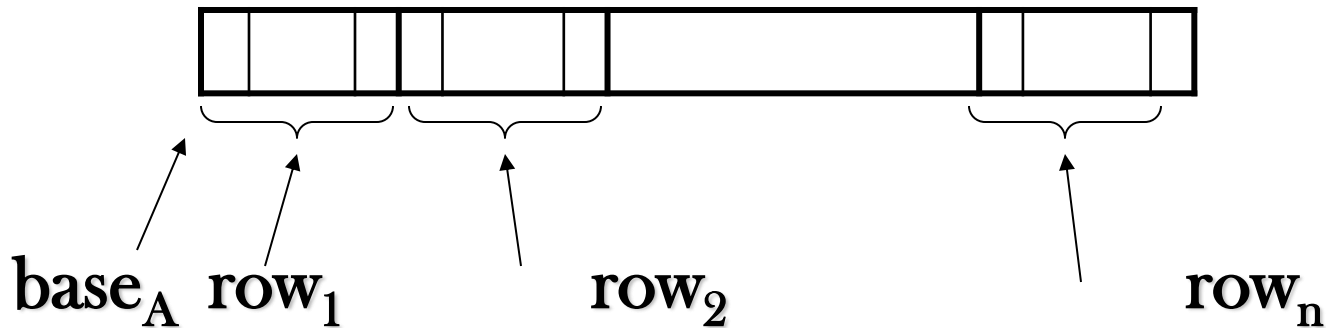
should be computed at run-time

can be computed at compile-time

- So, the location of $A[i]$ can be computed at the run-time by evaluating the formula $i * \text{width} + c$ where c is $(\text{base}_A - \text{low} * \text{width})$ which is evaluated at compile-time.
- Intermediate code generator should produce the code to evaluate this formula $i * \text{width} + c$ (one multiplication and one addition operation).

Two-Dimensional Arrays

- A two-dimensional array can be stored in
 - either row-major (*row-by-row*)
 - or column-major (*column-by-column*).
- Most of the programming languages use row-major method.
- Row-major representation of a two-dimensional array:



Two-Dimensional Arrays (cont.)

- The location of $A[i_1, i_2]$ is

$$\text{base}_A + ((i_1 - \text{low}_1) * n_2 + i_2 - \text{low}_2) * \text{width}$$

base_A is the location of the array A.

low_1 is the index of the first row

low_2 is the index of the first column

n_2 is the number of elements in each row

width is the width of each array element

- Again, this formula can be re-written as

$$\underbrace{((i_1 * n_2) + i_2) * \text{width}}_{\text{should be computed at run-time}} + \underbrace{(\text{base}_A - ((\text{low}_1 * n_1) + \text{low}_2) * \text{width})}_{\text{can be computed at compile-time}}$$

should be computed at run-time can be computed at compile-time

Multi-Dimensional Arrays

- In general, the location of $A[i_1, i_2, \dots, i_k]$ is

$$((\dots ((i_1 * n_2) + i_2) \dots) * n_k + i_k) * \text{width} + (\text{base}_A - ((\dots ((\text{low}_1 * n_1) + \text{low}_2) \dots) * n_k + \text{low}_k) * \text{width})$$

- So, the intermediate code generator should produce the codes to evaluate the following formula (to find the location of $A[i_1, i_2, \dots, i_k]$) :

$$((\dots ((i_1 * n_2) + i_2) \dots) * n_k + i_k) * \text{width} + c$$

- To evaluate the $((\dots ((i_1 * n_2) + i_2) \dots) * n_k + i_k)$ portion of this formula, we can use the recurrence equation:

$$e_1 = i_1$$

$$e_m = e_{m-1} * n_m + i_m$$

Translation Scheme for Arrays (P.483)

- If we use the following grammar to calculate addresses of array elements, we need inherited attributes.

$$L \rightarrow \text{id} \mid \text{id} [\text{Elist}]$$
$$\text{Elist} \rightarrow \text{Elist} , \text{E} \mid \text{E}$$

- Instead of this grammar, we will use the following grammar to calculate addresses of array elements so that we do not need inherited attributes (we will use only synthesized attributes).

$$L \rightarrow \text{id} \mid \text{Elist}]$$
$$\text{Elist} \rightarrow \text{Elist} , \text{E} \mid \text{id} [\text{E}$$

Translation Scheme for Arrays (cont.)

$S \rightarrow L := E$ { if (L.offset is null) emit('mov' L.place ',', E.place)
 else emit('mov' L.place '[' L.offset ']' ',', E.place) }

$E \rightarrow E_1 + E_2$ { E.place = newtemp();
 emit('add' E.place ',', E₁.place ',', E₂.place) }

$E \rightarrow (E_1)$ { E.place = E₁.place; }

$E \rightarrow L$ { if (L.offset is null) E.place = L.place)
 else { E.place = newtemp();
 emit('mov' E.place ',', L.place '[' L.offset ']') } }

Translation Scheme for Arrays (cont.)

$L \rightarrow id \quad \{ L.place = id.place; L.offset = null; \}$

$L \rightarrow Elist \mid$

$\{ L.place = newtemp(); L.offset = newtemp();$
 $\text{emit}('mov' \ L.place \ ',', c(Elist.array));$
 $\text{emit}('mult' \ L.offset \ ',', Elist.place \ ',', width(Elist.array)) \}$

$Elist \rightarrow Elist_1, E$

$\{ Elist.array = Elist_1.array; Elist.place = newtemp(); Elist.ndim = Elist_1.ndim + 1;$
 $\text{emit}('mult' \ Elist.place \ ',', Elist_1.place \ ',', limit(Elist.array, Elist.ndim));$
 $\text{emit}('add' \ Elist.place \ ',', Elist.place \ ',', E.place); \}$

$Elist \rightarrow id \mid E$

$\{ Elist.array = id.place; Elist.place = E.place; Elist.ndim = 1; \}$

Translation Scheme for Arrays - Example1

- A one-dimensional double array A : 5..100
→ $n_1=95$ width=8 (double) $low_1=5$
- Intermediate codes corresponding to $x := A[y]$

mov t1,,c ; where $c = \text{base}_A - (5) * 8$

mult t2,y,8

mov t3,,t1[t2]

mov x,,t3

Translation Scheme for Arrays - Example2

- A two-dimensional int array A : 1..10x1..20
 $\rightarrow n_1=10 \quad n_2=20 \quad \text{width}=4 \text{ (integers)} \quad \text{low}_1=1 \quad \text{low}_2=1$
- Intermediate codes corresponding to $x := A[y,z]$

```
mult t1,y,20                ; ((i1 * n2)+i2) * width
add t1,z,t1
mov t2,,c                    ;where c=baseA-(1 * 20+1) * 4
mult t3,t1,4
mov t4,,t2[t3]
mov x,,t4
```

Translation Scheme for Arrays – Example3

- A three-dimensional int array A : 0..9x0..19x0..29
 $\rightarrow n_1=10 \quad n_2=20 \quad n_3=30 \quad \text{width}=4 \text{ (integers)} \quad \text{low}_1=0 \quad \text{low}_2=0 \quad \text{low}_3=0$
- Intermediate codes corresponding to $x := A[w,y,z]$

mult t1,w,20 ; (((i₁*n₂)+i₂)*n₃+i₃)*width

add t1,y,t1

mult t2,t1,30

add t2,z,t2

mov t3,,c ; where $c = \text{base}_A - ((0 * 20 + 0) * 30 + 0) * 4$

mult t4,t2,4

mov t5,,t3[t4]

mov x,,t5

Boolean Expressions

$E \rightarrow E_1 \text{ and } E_2$

{ E.place = newtemp(); emit('and' E.place ',', E₁.place ',', E₂.place)

$E \rightarrow E_1 \text{ or } E_2$

{ E.place = newtemp(); emit('or' E.place ',', E₁.place ',', E₂.place)

$E \rightarrow \text{not } E_1$

{ E.place = newtemp(); emit('not' E.place ',', E₁.place) }

$E \rightarrow E_1 \text{ relop } E_2$

{ E.place = newtemp(); emit(relop.code E.place ',', E₁.place ',', E₂.place) }

Translation Scheme(cont.)

```
S → while { E.inloc = S.inloc } E do
    { emit(E.outloc 'jmpf' E.place ',', 'NOTKNOWN');
      S1.inloc=E.outloc+1; } S1
    { emit(S1.outloc 'jmp' ',', S.inloc);
      S.outloc=S1.outloc+1;
      backpatch(E.outloc,S.outloc); }
```

```
S → if { E.inloc = S.inloc } E then
    { emit(E.outloc 'jmpf' E.place ',', 'NOTKNOWN');
      S1.inloc=E.outloc+1; } S1 else
    { emit(S1.outloc 'jmp' ',', 'NOTKNOWN');
      S2.inloc=S1.outloc+1;
      backpatch(E.outloc,S2.inloc); } S2
    { S.outloc=S2.outloc;
      backpatch(S1.outloc,S.outloc); }
```

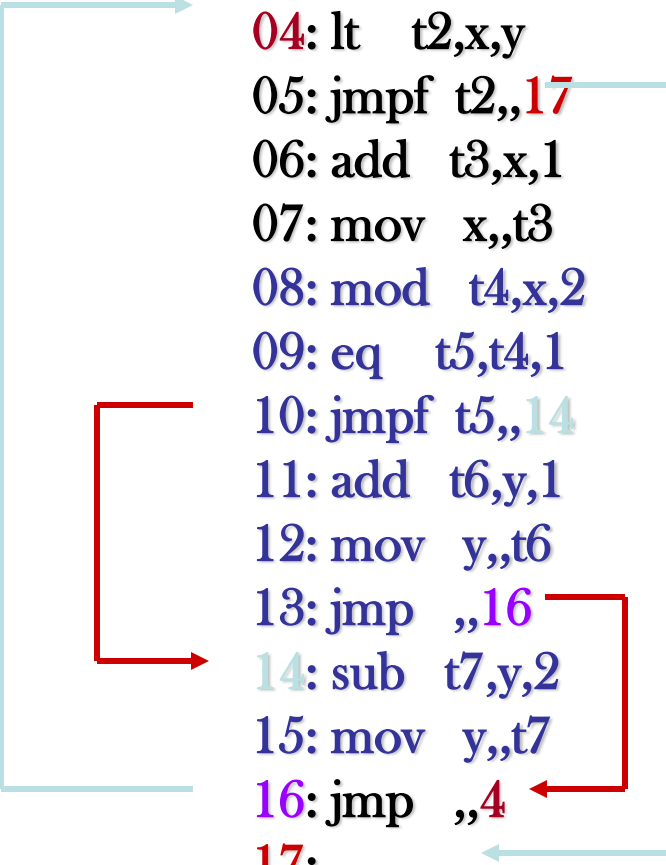
Three Address Codes - Example

```

x:=1;
y:=x+10;
while (x<y) {
    x:=x+1;
    if (x%2==1) then y:=y+1; →
    else y:=y-2;
}
    
```

```

01: mov  x,,1
02: add  t1,x,10
03: mov  y,,t1
04: lt   t2,x,y
05: jmpf t2,,17
06: add  t3,x,1
07: mov  x,,t3
08: mod  t4,x,2
09: eq   t5,t4,1
10: jmpf t5,,14
11: add  t6,y,1
12: mov  y,,t6
13: jmp  ,,16
14: sub  t7,y,2
15: mov  y,,t7
16: jmp  ,,4
17:
    
```



Exercises

- Translating the following C codes into three address codes.

```
int fun_for()
{
  int i,s=0;
  int a[10]={0,1,2,3,4,5,6,7,8,9};
  for (i=0;i<10;i++)
    s=s+a[i];
  return s;
}
```

```
int fun_if()
{
  int i=10,j=12;
  if(i<j)
    j=j-i;
  else
    i=i-j;
  return i+j;
}
```